

Sistema de Gestión de Tutorías Universitarias

Universidad de Especialidades Espíritu Santo · Diseño de Software | UCOMo310

Informe Técnico Oficial — Actividad 5 | AE3 – Incremento 1 del Proyecto Integrador. Integración de Diseño Orientado a Objetos, Patrones Estructurales y de Comportamiento.

Estudiante: Joffre Barre Veliz

Correo: joffre.barre@uees.edu.ec

Profesor / Tutor: Ph.D. Jaime Paul Sayago Heredia · PEL 4 –
2026

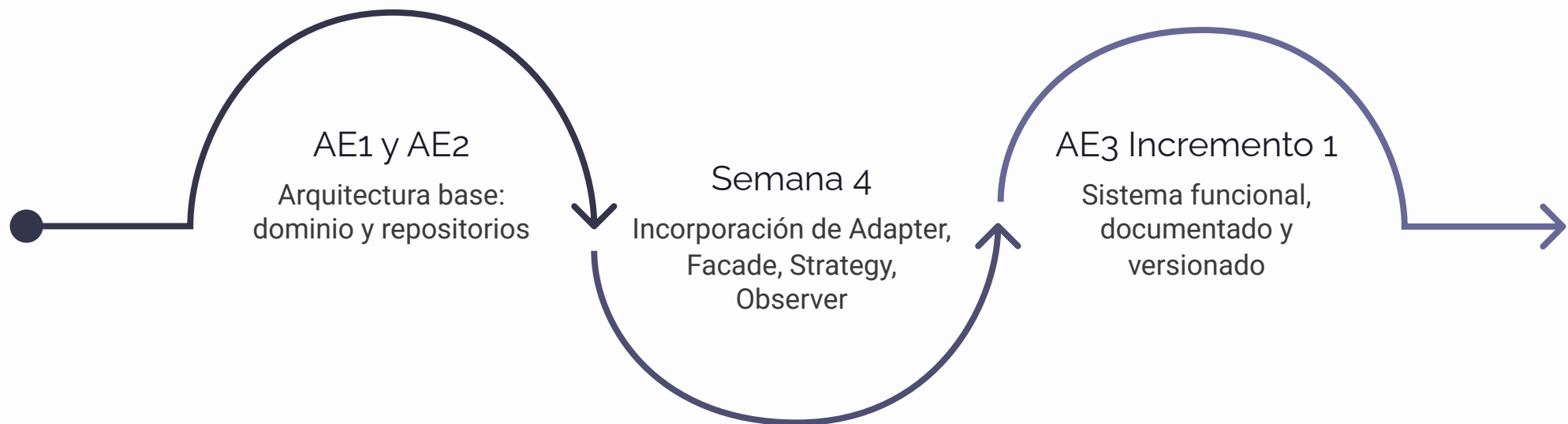
Fecha de Entrega: 8 de Septiembre de 2026

Repositorio:

github.com/joffrebarrev/Sistema_gestion_tutorias

1. Propósito del Incremento

La Evaluación Acreditable 3 (AE3) consolida el primer incremento funcional, documentado y versionado del proyecto integrador Sistema de Gestión de Tutorías Universitarias. Este incremento evoluciona la arquitectura base diseñada en AE1 y AE2 mediante la incorporación integrada de patrones de diseño de la Semana 4.



Los cuatro patrones incorporados — Adapter, Facade, Strategy y Observer — responden a problemas reales de acoplamiento, variabilidad, integraciones externas y orquestación de servicios identificados en el proyecto.

2. Estado Inicial del Proyecto

2.1 Dominio Base — AE1

Paquete edu.uees.tutorias.domain con las entidades clave del sistema:

- **Estudiante, Docente y Usuario:** Modelado de actores con herencia/composición clara
- **Reserva y EstadoReserva:** Gestión de estados PENDIENTE, CONFIRMADA, CANCELADA
- **HorarioTutoria y Asignatura:** Franjas horarias y materias
- **RepositorioReservas:** Abstracción de persistencia aplicando DIP

2.2 Análisis Comparativo de Patrones Creacionales (AE2)

Se reevaluó la permanencia de los patrones creacionales para mantener un diseño magro y evitar sobreingeniería:

Patrón	Decisión
Factory Method	✓ Se Mantiene — Instancia dinámicamente canales de comunicación sin acoplar el creador, integrándose con los observadores
Builder	✗ Simplificado — Los atributos de Reserva son obligatorios y bien delimitados; la orquestación la asume TutoriasFacade

3. Problemas Reales de Diseño Identificados

1

Incompatibilidad con Plataformas Externas

APIs de Zoom (`crearReunion()`) y Microsoft Teams (`agendarVideollamada()`) poseen métodos incompatibles entre sí. Invocarlas directamente acoplaba el dominio a librerías de terceros.

2

Complejidad en Tutorías Virtuales

Para agendar una tutoría virtual, el cliente debía coordinar 4 subsistemas independientes: validación, agendamiento, generación de enlace y emisión de notificaciones.

3

Violación del OCP en Cancelaciones

Reglas de reembolso para cancelaciones (Normal, Prioritaria, Grupal, Emergencia) requerían múltiples condicionales `if/else`. Cualquier cambio en la política alteraba la clase central de servicio.

4

Dependencias Directas en Notificación

Al cambiar el estado de una tutoría, la entidad de dominio invocaba directamente correo, calendario y paneles web, violando la cohesión y el principio SRP.

4. Justificación Integrada de Patrones — Estructurales

Análisis comparativo de los dos patrones estructurales aplicados en el Incremento 1:

Elemento	Patrón 1: Adapter	Patrón 2: Facade
Problema Real	Incompatibilidad de nombres y métodos en SDKs externos de videollamada	El cliente conoce demasiados detalles y pasos para agendar una tutoría
Contexto	Integración de proveedores Zoom y Teams	Orquestación del flujo principal en <code>TutoriasFacade</code>
Qué Cambia	El proveedor de videoconferencia seleccionado	La secuencia interna de invocación de los subsistemas
Qué Permanece	La interfaz uniforme <code>Videoconferencia</code>	La operación de alto nivel <code>crearTutoriaVirtual()</code>
Principio SOLID	DIP & OCP	SRP & Law of Demeter
Beneficio	Intercambiabilidad transparente de salas virtuales	Interfaz simplificada de un solo punto de entrada para el cliente
Costo	Creación de clases adaptadoras intermedias	La fachada puede volverse un punto central si no se delimita

4. Justificación Integrada de Patrones — Comportamiento

Análisis comparativo de los dos patrones de comportamiento aplicados en el Incremento 1:

Elemento	Patrón 3: Strategy	Patrón 4: Observer
Problema Real	Políticas de cancelación variables con condicionales complejas	Múltiples componentes deben reaccionar al cambio de estado de reserva
Contexto	Proceso de validación de cancelación de tutorías	Notificación automática al confirmar o cancelar reservas
Qué Cambia	Las reglas temporales de cancelación (24h, 2h, 1h, 15m)	La cantidad y comportamiento de canales/paneles suscritos
Qué Permanece	El contrato <code>PoliticaCancelacion</code> y el flujo de <code>ServicioCancelacion</code>	El sujeto emisor <code>Reserva</code> y la interfaz <code>ReservaObserver</code>
Principio SOLID	OCP & DIP	SRP & OCP
Beneficio	Adición de reglas de reembolso sin alterar código probado	Desacoplamiento total entre el dominio y los medios de difusión
Cómo se Verifica	Pasar reservas con distintas horas restantes y evaluar la excepción esperada	Impresión unificada en consola de las 4 respuestas de observadores

5. Decisiones de Diseño SOLID



Single Responsibility Principle (SRP)

Reserva gestiona el ciclo de vida del dominio; TutoriasFacade orquesta el flujo de servicios; los observadores manejan exclusivamente la salida hacia sus respectivos canales. Cada clase tiene una única razón para cambiar.



Open/Closed Principle (OCP)

Se pueden integrar nuevos proveedores de videollamada (ej. Google Meet) mediante un nuevo Adapter, o añadir nuevas reglas de cancelación mediante Strategy, **sin modificar el código fuente existente**.



Dependency Inversion Principle (DIP)

Todos los servicios principales dependen de abstracciones e interfaces (Videoconferencia, PoliticaCancelacion, ReservaObserver), anulando el acoplamiento a clases concretas.

6. Diagrama UML Completo del Incremento 1

@startuml

skinparam classAttributeIconSize 0

```
package "edu.uees.disenosoftware.domain" {
    enum EstadoReserva { PENDIENTE CONFIRMADA CANCELADA }
    class Reserva {
        - id: String
        - fechaHora: LocalDateTime
        - estado: EstadoReserva
        - observers: List<ReservaObserver>
        + agregarObserver(observer: ReservaObserver): void
        + confirmar(): void
        + cancelar(): void
        + horasRestantes(): long
    }
}

package "edu.uees.disenosoftware.adapter" {
    interface Videoconferencia {
        + crearSesion(titulo: String, fechaHora: LocalDateTime): String
    }
    class ZoomAdapter implements Videoconferencia
    class TeamsAdapter implements Videoconferencia
    class ProveedorZoom { + crearReunion(topic: String): String }
    class MicrosoftTeamsAPI { + agendarVideollamada(subject:
String): String }
    ZoomAdapter --> ProveedorZoom
    TeamsAdapter --> MicrosoftTeamsAPI
}

package "edu.uees.disenosoftware.strategy" {
    interface PoliticaCancelacion {
        + puedeCancelar(reserva: Reserva): boolean
    }
    class CancelacionNormal implements PoliticaCancelacion
    class CancelacionPrioritaria implements PoliticaCancelacion
    class CancelacionGrupal implements PoliticaCancelacion
    class CancelacionEmergencia implements PoliticaCancelacion
    class ServicioCancelacion {
        - politica: PoliticaCancelacion
        + cancelar(reserva: Reserva): void
    }
    ServicioCancelacion --> PoliticaCancelacion
}

package "edu.uees.disenosoftware.observer" {
    interface ReservaObserver { + actualizar(reserva: Reserva): void }
    class EmailObserver implements ReservaObserver
    class CalendarioObserver implements ReservaObserver
    class PanelEstudianteObserver implements ReservaObserver
    class PanelDocenteObserver implements ReservaObserver
}

package "edu.uees.disenosoftware.facade" {
    class TutoriasFacade {
        - servicioReservas: ServicioReservas
        - videoconferencia: Videoconferencia
        + crearTutoriaVirtual(...): String
    }
}

Reserva --> EstadoReserva
Reserva "1" o-- "0..*" ReservaObserver
TutoriasFacade --> Videoconferencia
TutoriasFacade ..> Reserva
ServicioCancelacion ..> Reserva
PoliticaCancelacion ..> Reserva
@enduml
```

Descripción de Paquetes

- **Domain:** Entidades base (Reserva, EstadoReserva) y el patrón Observer
- **Adapter:** Adaptadores para Zoom y Teams con interfaz unificada Videoconferencia
- **Strategy:** Políticas de cancelación intercambiables
- **Facade:** Orquestador central TutoriasFacade
- **Relaciones:** Dependencias inversas hacia interfaces, composición clara entre componentes

☐ Las dependencias (--> y ..>) apuntan siempre hacia abstracciones, nunca hacia clases concretas — aplicando DIP de forma consistente.

7 & 8. Verificación de Compilación y Trazabilidad Git

Output de Apache Maven — BUILD SUCCESS

```
[INFO] Scanning for projects...
[INFO] -----
[INFO] Building Sistema_gestion_tutorias 1.0-SNAPSHOT
[INFO] -----
[INFO] --- maven-resources-plugin:3.3.1:resources ---
[INFO] --- maven-compiler-plugin:3.13.0:compile ---
[INFO] Compiling 26 source files to target\classes
[INFO] -----
[INFO] BUILD SUCCESS
[INFO] -----
[INFO] Total time: 6.392 s
[INFO] Finished at: 2026-09-07T11:11:41-05:00
[INFO] -----
```

26

Source Files

Compilados exitosamente

6.3s

Build Time

Tiempo total de compilación

Historial de Commits — Incremento 1

01	refactor: mejorar responsabilidades del dominio y limpiar dependencias
02	feat: integrar proveedores externos de videoconferencia con adapter
03	feat: simplificar flujo de agendamiento con fachada tutoriasfacade
04	feat: aplicar estrategia a políticas variables de cancelación
05	feat: implementar notificaciones descentralizadas con observer
06	docs: actualizar UML del Incremento 1 y README oficial

9. Conclusiones del Incremento 1

Se consolidó exitosamente el primer incremento del proyecto integrador acoplando armónicamente patrones creacionales, estructurales y de comportamiento en el Sistema de Gestión de Tutorías Universitarias.

Adapter + Facade

Simplificaron radicalmente la complejidad del sistema hacia clientes y plataformas de terceros, unificando interfaces heterogéneas en un solo punto de entrada.

Strategy + Observer

Aislaron las variaciones de reglas de negocio y canales de comunicación, respetando el principio Abierto/Cerrado (OCP) en todos los puntos de extensión.

Arquitectura Resultante

Extensible, mantenible y desacoplada. Permite futuras integraciones — nuevos proveedores, nuevas políticas, nuevos canales — sin modificar código probado.

Se declara que se utilizó asistencia de Inteligencia Artificial como herramienta interactiva para la estructuración y diagramación en PlantUML de este informe técnico, habiendo comprendido, validado y respaldado en su totalidad el código fuente, la arquitectura y las decisiones de diseño presentadas en esta evaluación.